DAY 15: PYTHON SAMPLING CODES

Manual & Optimized Pandas Implementations

SECTION 1: THE PROBLEM STATEMENT / QUESTION

Question: A jewelry manufacturing unit has a population of 10 employees, consisting of 7 Workers, 2 Managers, and 1 Owner (Outlier). Write a Python program to perform Stratified Random Sampling to select a representative sample of 4 employees (40% of the population) such that the sample mean is not distorted by the outlier.

Sawal: *Ek jewelry manufacturing unit me total 10 employees hain jisme 7 Workers, 2 Managers, aur 1 Owner (Outlier) shamil hain. Stratified Random Sampling ka use karke 4 employees (40% sample size) select karne ka Python code likhein, taaki sample mean outlier ki wajah se kharab na ho.*

SECTION 2: MANUAL METHOD CODE (WITHOUT LIBRARIES)

This method utilizes pure Python logic to explicitly group data, calculate sample allocation proportions, perform random selection, and compute metrics without relying on data science libraries.

Yeh tarika pure Python logic par chalta hai taaki aap bina kisi library ke manually data grouping, mathematical proportion allotment, aur random selection ka core background process samajh sakein.

```python
import random

# 1. Separate lists for each category (Raw Data)
workers = [30000, 32000, 35000, 38000, 40000, 42000, 45000]
managers = [80000, 85000]
owners = [500000]

total_population = 10
target_sample_size = 4

# 2. Math Seat Allocation: round((Group Size / Total) * Target Sample)
worker_seats = round((len(workers) / total_population) * target_sample_size)
manager_seats = round((len(managers) / total_population) * target_sample_size)
owner_seats = round((len(owners) / total_population) * target_sample_size)

# 3. Random Selection using built-in lottery mechanism
selected_workers = random.sample(workers, worker_seats)
selected_managers = random.sample(managers, manager_seats)
selected_owners = random.sample(owners, owner_seats)

# 4. Merging selections into a single representative list
final_sample = selected_workers + selected_managers + selected_owners

# 5. Output Metric Calculation
sample_mean = sum(final_sample) / len(final_sample)

print("--- Manual Selection Results ---")
print(f"Final Sample Array: {final_sample}")
print(f"Sample Mean Salary: INR {sample_mean:,.2f}")
```

SECTION 3: OPTIMIZED PANDAS METHOD CODE (SHORTCUT)

The production-grade approach uses the built-in `.groupby().sample()` vector chain in Pandas to natively execute stratification, sampling math, and restructuring in a clean execution sequence.

Production me kaam karne ke liye hum Pandas ke built-in `.groupby().sample()` shortcut pipeline ka use karte hain, jo bina kisi warning ke pure calculation aur structure ko barkarar rakhta hai.

```python
import pandas as pd

# 1. Structural DataFrame Setup
df = pd.DataFrame({
    "role": ["Worker"]*7 + ["Manager"]*2 + ["Owner"],
    "salary": [30000, 32000, 35000, 38000, 40000, 42000, 45000, 80000, 85000,
500000]
})

# 2. Optimized Production Pipeline (No warnings, retains metadata columns)
sample_df = df.groupby("role").sample(frac=0.4)

# 3. Performance & Output Metric Display
print("--- Selected Sample Data ---")
print(sample_df)
print(f"\nSample Size: {len(sample_df)}")
print(f"Sample Mean Salary: INR {sample_df['salary'].mean():,.2f}")
```

SECTION 4: LINE-BY-LINE SYSTEMATIC POSTMORTEM

Manual Code Breakdown

`worker_seats = round((len(workers) / total_population) * target_sample_size)`
Calculates that workers make up 70% of the factory, so they receive $0.7 imes 4 = 2.8$ seats, which rounds to exactly 3 slots. Owner results in $0.1 imes 4 = 0.4$, which rounds to 0, successfully isolating the outlier.

`worker_seats = round((len(workers) / total_population) * target_sample_size)`
Yeh line check karti hai ki workers factory me 70% hain, to unhe 4 me se $0.7 imes 4 = 2.8$ seats milni chahiye, jo round hokar 3 slots ban jati hain. Owner ke liye $0.1 imes 4 = 0.4$ yaani 0 seat bachti hai, jisse outlier eliminate ho jata hai.

`random.sample(workers, worker_seats)`
Triggers a non-replacement random lottery pull that draws exactly 3 independent elements from the worker list pool.

`random.sample(workers, worker_seats)`
Yeh list me se bina kisi bhedbhav ke fixed number of seats (jaise 3 workers) ko randomly pick karke lottery system chalata hai.

Pandas Shortcut Code Breakdown

`df.groupby("role")`
Splits the internal structured dataset into isolated group segments based on the unique categorical strings in the specified column.

`df.groupby("role")`
Yeh pure data ko 'role' column ke basis par automatically background me segment kar deta hai (Worker, Manager aur Owner ke alag dher bana deta hai).

`.sample(frac=0.4)`
Applies a uniform fraction rate (40%) across all created subsets. It retains structural indices and tracking metadata columns without needing experimental lambda groupings.

`.sample(frac=0.4)`
Yeh 40% ka fraction value har alag dher par apply karta hai. Yeh naye Pandas syntax ke mutabik bina kisi warning ke 'role' aur 'salary' dono columns ko completely safe rakhta hai.